Individual Project Prototype

Overview

Using React and Node.js Express, we can build a ChatGPT Clone.

1. It uses React Frontend and Node.js Express Server.
2. The Express Server uses ChatGPT (OpenAI) API.
3. This project prototype has seven steps.

This app is a prototype of an individual project.

Seven Steps

1. Frontend UI React
2. Express Server with WebSocket
3. Sessions
4. Sending Messages from Server
5. Working with OpenAI
6. Deployment
7. Standalone ChatGPT Clone

Code Directory

Each step is a complete project in the `code` directory: every step has its own `run.sh` and `README.md`.

```
code/
├── step1/  Frontend UI React (client)
├── step2/  Express Server with WebSocket (client, server)
├── step3/  Sessions
├── step4/  Sending Messages from Server
├── step5/  Working with OpenAI API
├── step6/  Deployment (server, electron)
└── step7/  Standalone ChatGPT Clone
```

```
chmod +x step*/run.sh   # only once
./step5/run.sh          # starts the server and the client
```

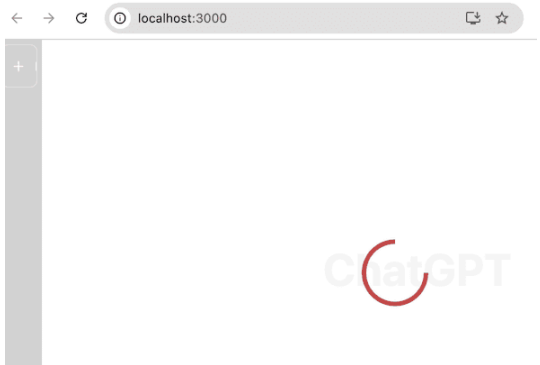
Running ChatGPT Clone

Open `step5` in the code directory: `./step5/run.sh` starts both the server and the client. To start them one by one:

Start the React Frontend (client)

Build and start React app: use a web browser to access http://localhost:3000/ if necessary.

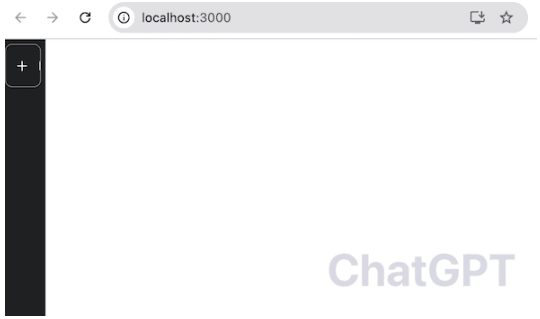
```
cd step5/client
npm install
npm start
```



Start the Node.js Backend (server)

Open `step5/server`, create `.env` (copy `.env.example` and set `OPENAI_API_KEY`), then build and start the server to connect to the client.

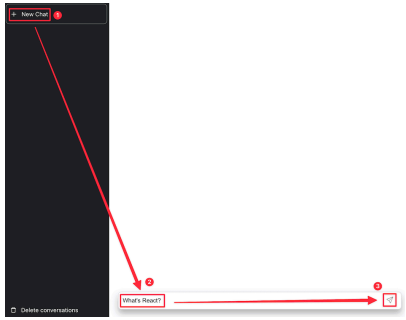
```
> npm install
> node index.js
App started listening at port 5000
user connected a87P3Ezy3Hb2cWXmAAAB
```



The 5000 is used for the communication port between client and server; change the port if necessary (will be explained later).

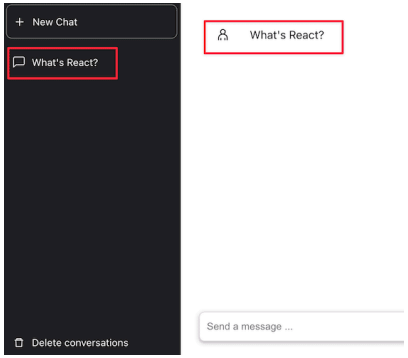
Start a Session

1. Click the "+ New Chat" button, and User Input will pop up.

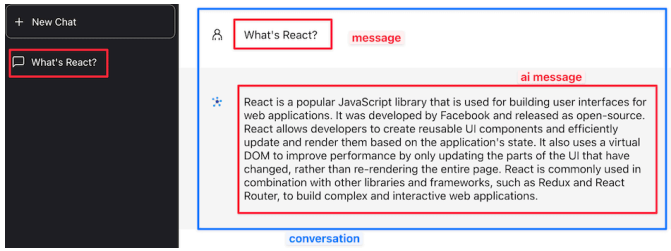


2. Give inputs (any questions to the ChatGPT clone)

- A message will be generated and sent to the server.

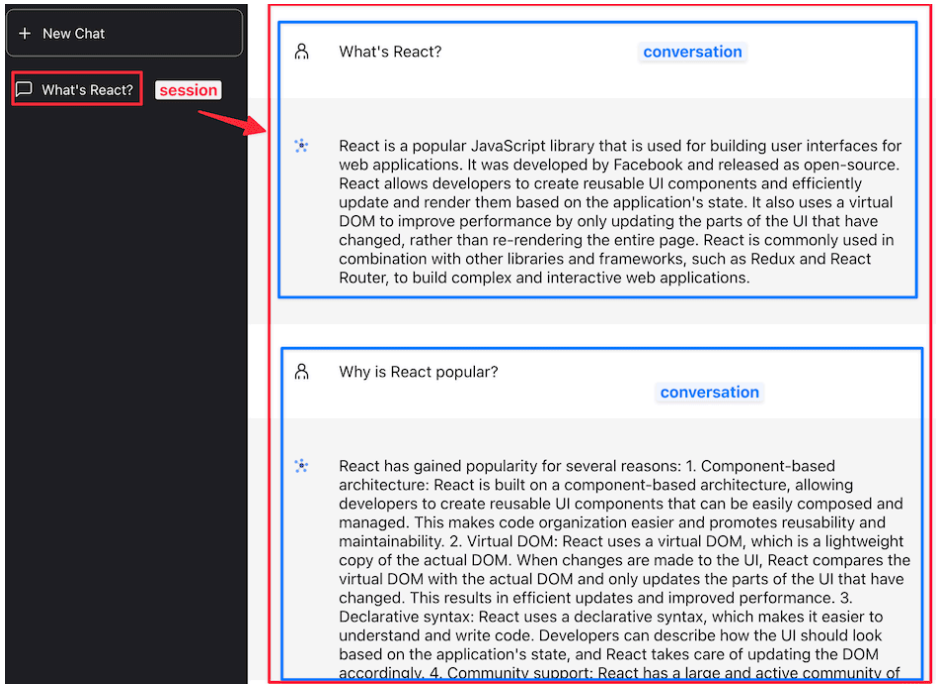


3. The server will use the ChatGPT API to ask the question to ChatGPT and get answers from the ChatGPT API.
4. The answer from ChatGPT will be returned to the client and displayed.



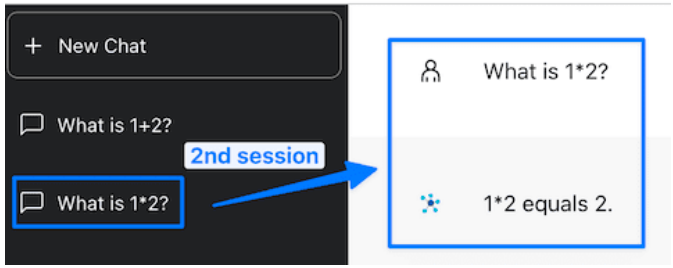
- The two messages (user message and AI message) are one conversation.

5. A session can have multiple conversations.



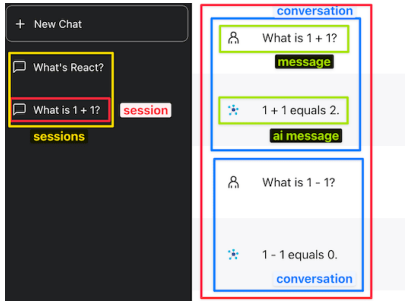
Start another Session

1. Click the "+ New Chat" button to start a `new session`.



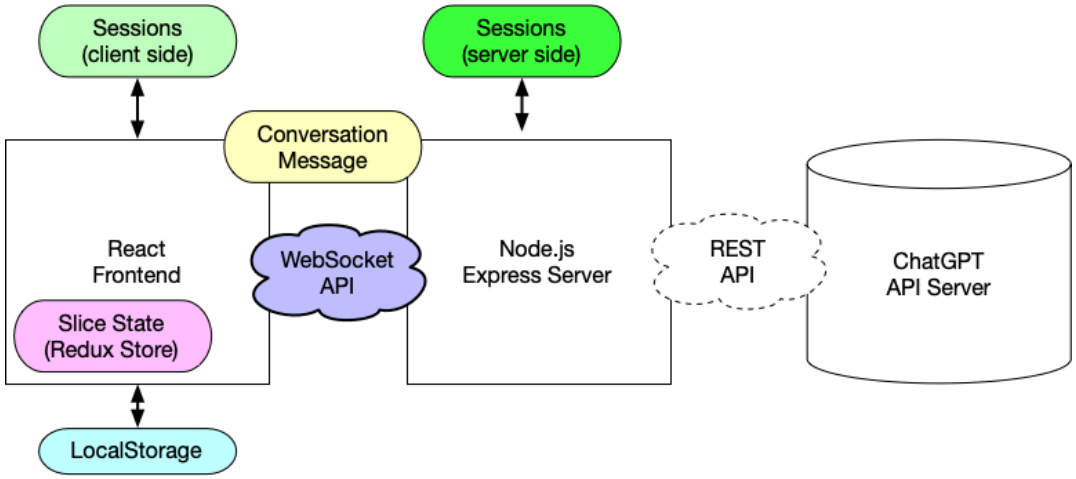
- The following conversations will be a part of the new session.

2. We have multiple sessions in this example.



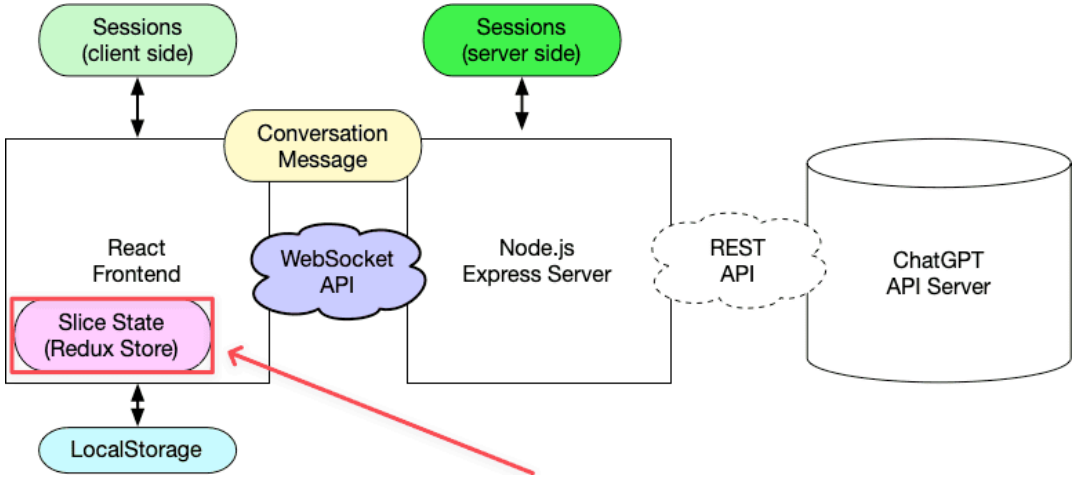
Data Structures for ChatGPT Clone Application

To manage sessions, messages, and conversations, we need to use multiple data structures (or data models).



Redux Store (Slice State): Dashboard/dashboardSlice.js

We use Redux Store (Slice State) to store and retrieve states per slice for the client: in other words, we use them for managing states/actions for the React frontend app.



States

```
const initialState = {
  sessionEstablished: false,
  conversations: [],
  selectedConversationId: null,
};
```

- `sessionEstablished` becomes true when the client receives `session-details` from the server (then the loading spinner disappears)

Reducers (Actions)

Format:

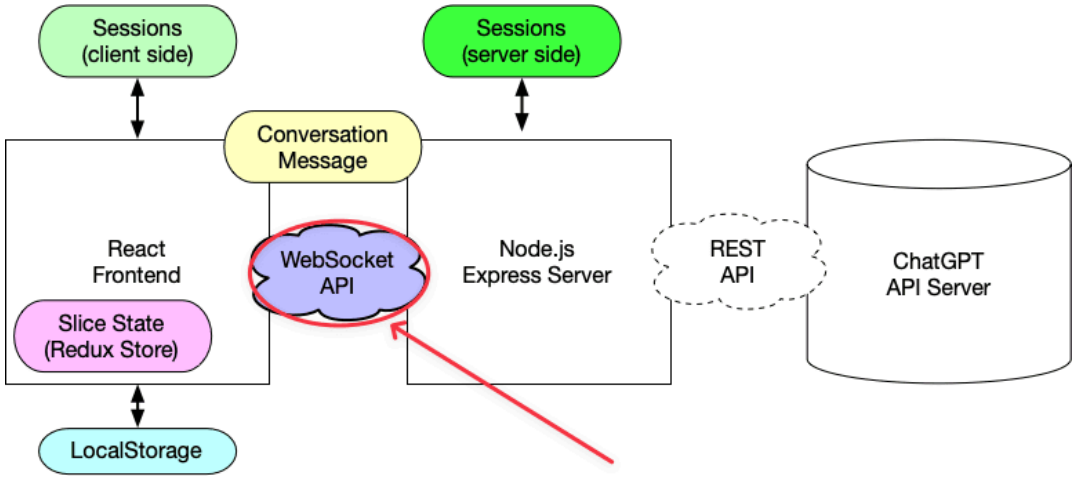
```
- reducer name: arguments
  - algorithm (how states are updated)
```

- setSelectedConversationId: conversationId
  - selectedConversationId <- conversationId
- addMessage: message, conversationId
  - if the conversation with conversationId exists: conversation.messages.push(message)
  - otherwise: conversations.push({id: conversationId, messages: [message]})
- setConversations: conversations
  - conversations <- conversations

- setConversationHistory: {id, messages}
    - if the conversation with id exists:

      conversation.messages <- messages
    - otherwise: conversations.push({id, messages})
- deleteConversations: delete conversations
    - conversations <- []
    - selectedConversationId <- null

WebSocket API



WebSocket API Protocols

- session-history
  - client emits "session-history" API with argument {sessionId}
  - server finds conversations = sessions[sessionId]
  - server emits "session-details" API with argument `session = {sessionId, conversations}`

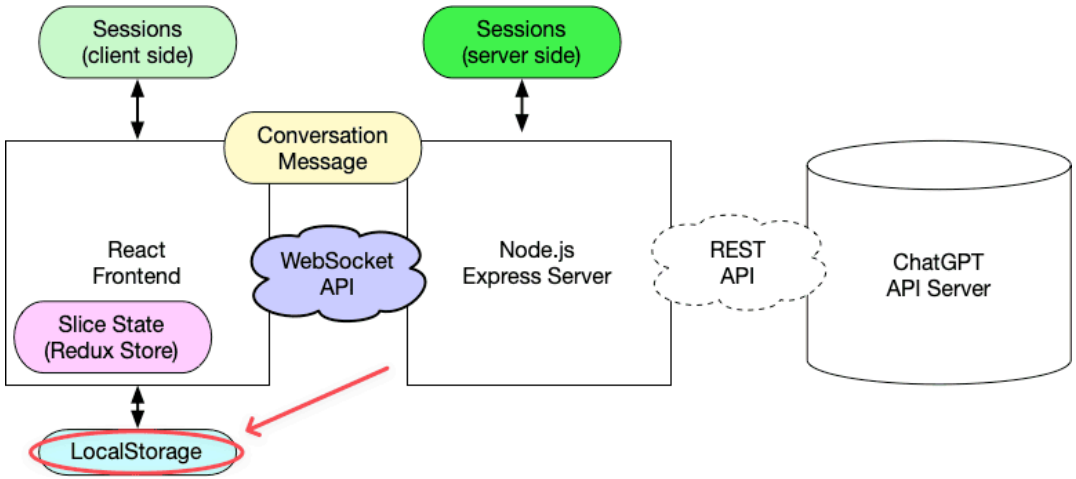
- conversation-message
  - client emits "conversation-message" API with argument {sessionId, message, conversationId}
  - server finds existing conversation in conversations = sessions[sessionId] from conversationId.
  - server invokes ChatGPT API using `message`.
  - server emits "conversation-details" API with argument `conversation = {id, messages: [message, aiMessage]}`.

- conversation-delete
  - client emits "conversation-delete" API with argument {sessionId}
  - server sets `sessions[sessionId] = []`;
- session-details
  - server emits "session-details" API with argument `{sessionId, conversations}`
  - client stores `localStorage.setItem("sessionId", sessionId)`
  - client calls reducer `setConversations(conversations)`

- conversation-details
  - server emits "conversation-details" API with argument `conversation`
  - client calls reducer `setConversationHistory(conversation)`

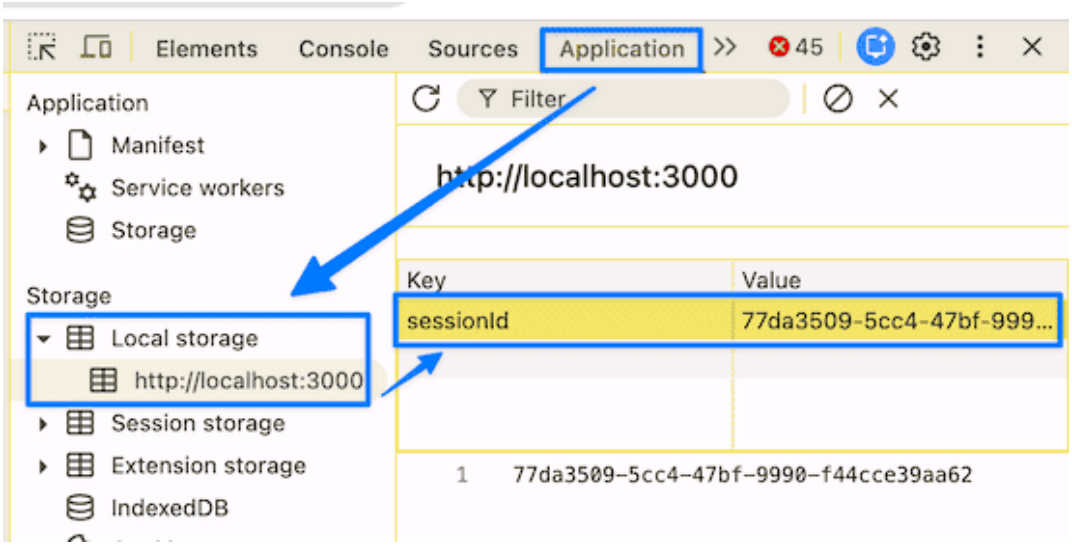
Local Storage (IndexedDB)

We use Local Storage (IndexedDB) to store/retrieve the current session ID.



To access Local Storage, we should use the Developer mode in our web browser. This is an example using Chrome.

- F12 for Developer mode.
- Choose Application & Local Storage.



We get the current session ID.

```
localStorage.getItem("sessionId")
```

Conversation Message

Message

We use Conversation Message JSON for communication between the client and the server.

- It has the information of
    - current session ID of current session
    - message (that users gave)
    - conversationId of this conversation

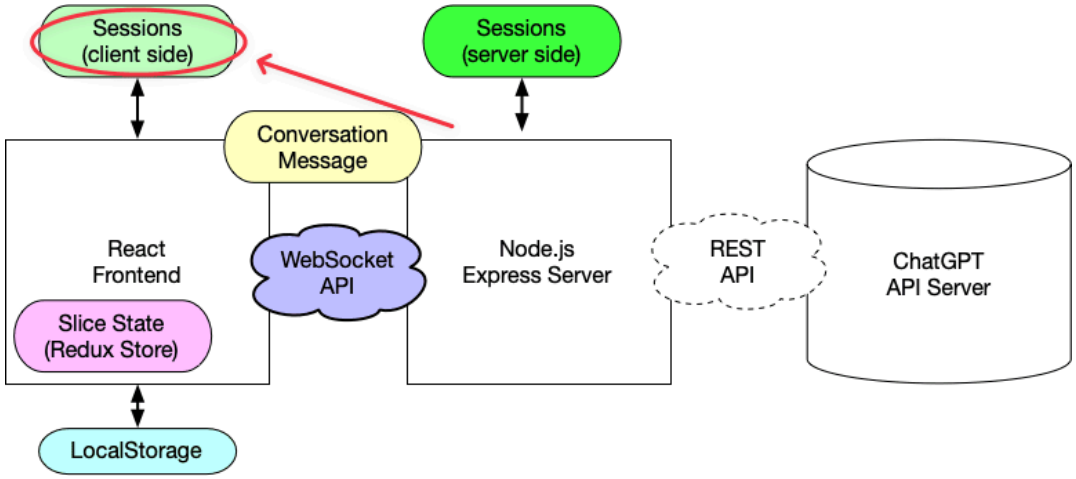
```
const message = {
   aiMessage: true/false,
   content,
   id: uuid(),
   animate: true/false, // optional
 };
```

```
const aiMessage = {
    aiMessage: true,
    content: aiMessageContent,
    id: uuid(),
  };
```

Conversation

```
{
  id: conversationId,
  messages: [message, aiMessage],
}
```

Session (Client Side)

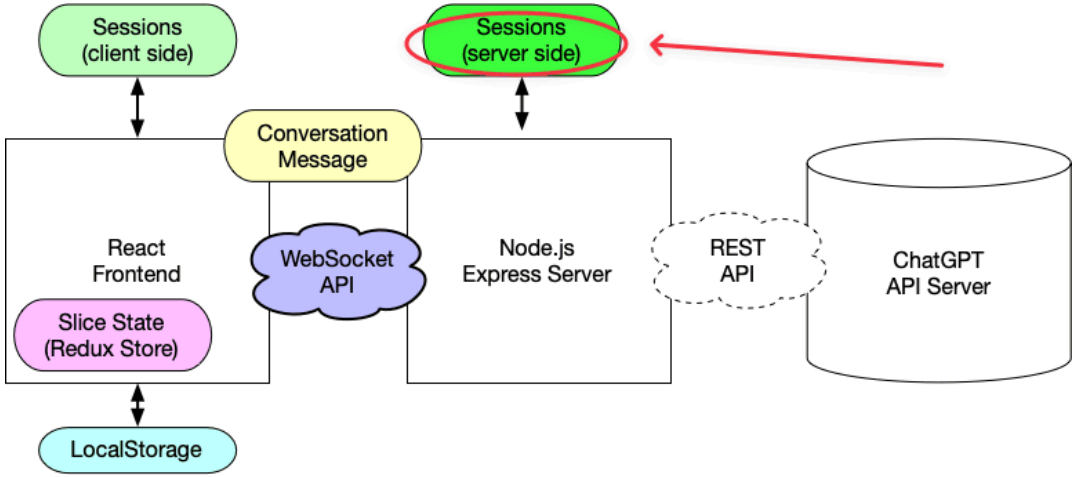


The server should maintain `sessions` to track the communication messages between the server and client.

- Each session has a sessionId
- The sessions dictionary keeps track of sessions using this sessionId

```
let sessions = {};
sessions[sessionId]
```

Session (Server Side)




```
sessions = {
  sessionId: UID,
  conversations: [conversation]
}
conversation = {
  id: UID,
  messages: [message, aiMessage],
}
message = {
 id: UID,
 content: string,
```

The message will be sent to the server and stored on the server as well.

```
user connected 1FIKMq8D3zgY2P3-AAAD
message came from client side
{
  message: {
    aiMessage: false,
    content: "What's React?",
    id: '18e931e2-5c90-433f-baca-1e825abc3219',
    animate: false
  },
  conversationId: '94be5564-1ff4-452f-8c41-41c59401a4f7'
}
```